



An-Najah National University
Computer Engineering department
Distributed Operating Systems
Turning the Bazar into an Amazon
Design document

Raheeq Qassrawi

Aya Sawalha

To test our full system (frontend + replicated services + caching + consistency), we follow these steps:

Build and Run the System with Docker Compose

Open a terminal in the project root directory and run:

✓ `docker-compose up -build`

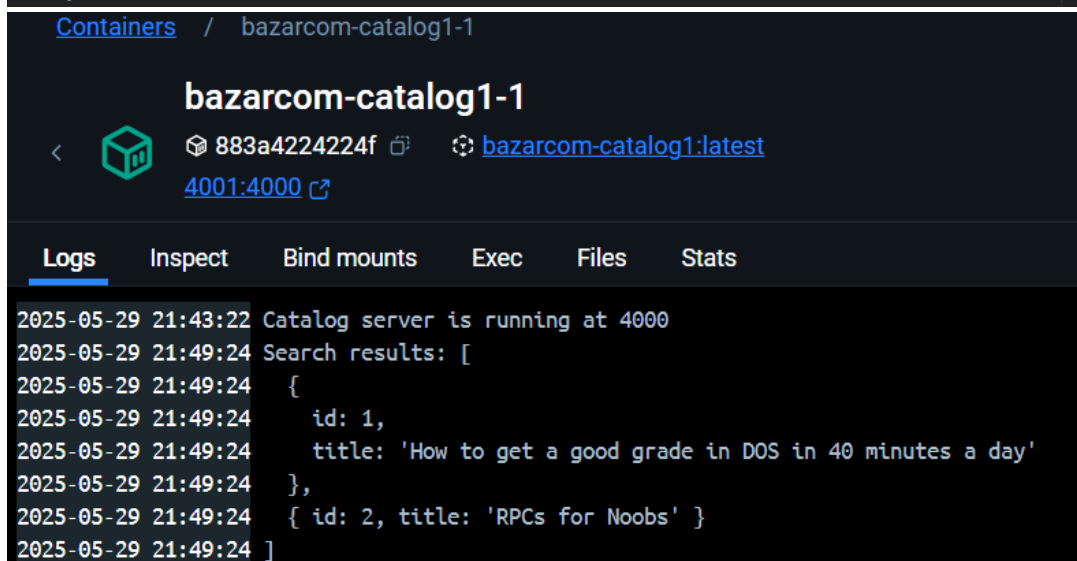
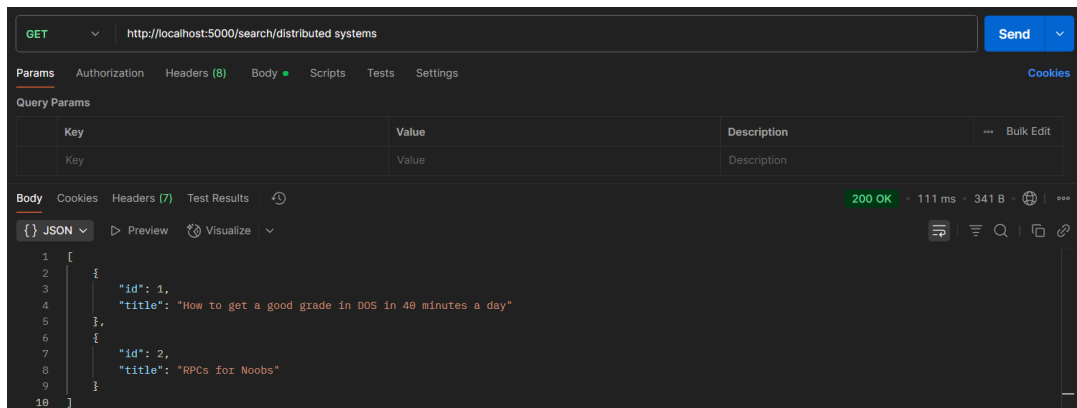
This command will:

- Build all services: `catalog1`, `catalog2`, `order1`, `order2`, `frontend`.
- Run them in isolated containers.
- Map ports so you can test them from Postman.

Test Frontend API (Port 5000)

Search for a topic:

✓ **GET** <http://localhost:5000/search/distributed%20systems>



Get info of a specific book:

✓ **GET** <http://localhost:5000/info/1>

The image shows two screenshots. The top screenshot is from a REST client (like Postman) showing a GET request to `http://localhost:5000/info/1` with a status of `200 OK`. The response body is a JSON object:

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "quantity": 14,
  "price": 30,
  "topic": "distributed systems"
}
```

The bottom screenshot is from the Docker Desktop interface, showing the logs for the container `bazarcom-catalog2-1`. The logs show the container starting and then receiving a request:

```
2025-05-29 21:43:22 Catalog server is running at 4000
2025-05-29 21:50:30 Item found: {
2025-05-29 21:50:30   id: 1,
2025-05-29 21:50:30   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 21:50:30   quantity: 14,
2025-05-29 21:50:30   price: 30,
2025-05-29 21:50:30   topic: 'distributed systems'
2025-05-29 21:50:30 }
```

Note: Each query appears on different catalog replica, so this means that the load balancing worked.

Purchase a book:

✓ **POST** <http://localhost:5000/purchase/1>

POST <http://localhost:5000/purchase/1> Send

Params Authorization Headers (8) Body • Scripts Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (7) Test Results 200 OK 230 ms 276 B

{ } JSON Preview Visualize

```
1 {
2   "message": "Book purchased successfully"
3 }
```

[Containers](#) / bazarcom-order1-1

bazarcom-order1-1

< 74430e5e1495 [bazarcom-order1:latest](#)
[3001:3000](#)

Logs Inspect Bind mounts Exec Files Stats

```
2025-05-29 21:43:22 Order service running at http://order:3000
2025-05-29 21:58:42 Book fetched successfully from catalog: {
2025-05-29 21:58:42   id: 1,
2025-05-29 21:58:42   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 21:58:42   quantity: 14,
2025-05-29 21:58:42   price: 30,
2025-05-29 21:58:42   topic: 'distributed systems'
2025-05-29 21:58:42 }
2025-05-29 21:58:42 Order saved: {
2025-05-29 21:58:42   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 21:58:42   newQuantity: 13,
2025-05-29 21:58:42   price: 30,
2025-05-29 21:58:42   state: 'succeeded'
2025-05-29 21:58:42 }
```

```
2025-05-29 21:58:42 you are inside catalog1 other replica is http://catalog2:4000
2025-05-29 21:58:42 Cache for book 1 invalidated on frontend.
2025-05-29 21:58:42 Item updated: {
2025-05-29 21:58:42   id: 1,
2025-05-29 21:58:42   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 21:58:42   quantity: 13,
2025-05-29 21:58:42   price: 30,
2025-05-29 21:58:42   topic: 'distributed systems'
2025-05-29 21:58:42 }
```

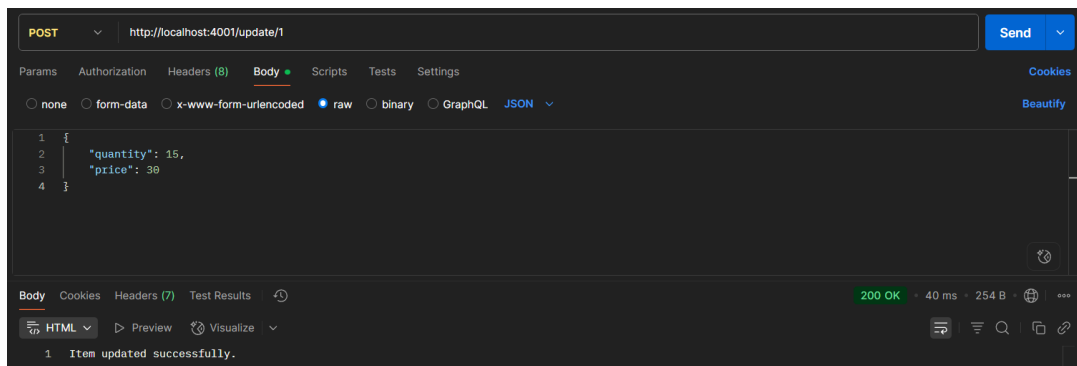
```
2025-05-29 21:58:42 you are inside catalog2 other replica is http://catalog1:4000
2025-05-29 21:58:42 Cache for book 1 invalidated on frontend.
2025-05-29 21:58:42 Item updated: {
2025-05-29 21:58:42   id: 1,
2025-05-29 21:58:42   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 21:58:42   quantity: 13,
2025-05-29 21:58:42   price: 30,
2025-05-29 21:58:42   topic: 'distributed systems'
2025-05-29 21:58:42 }
```

Note: The synchronization between catalog1 and catalog2 is working correctly.

Verify Replication and Consistency

Update a book in catalog1:

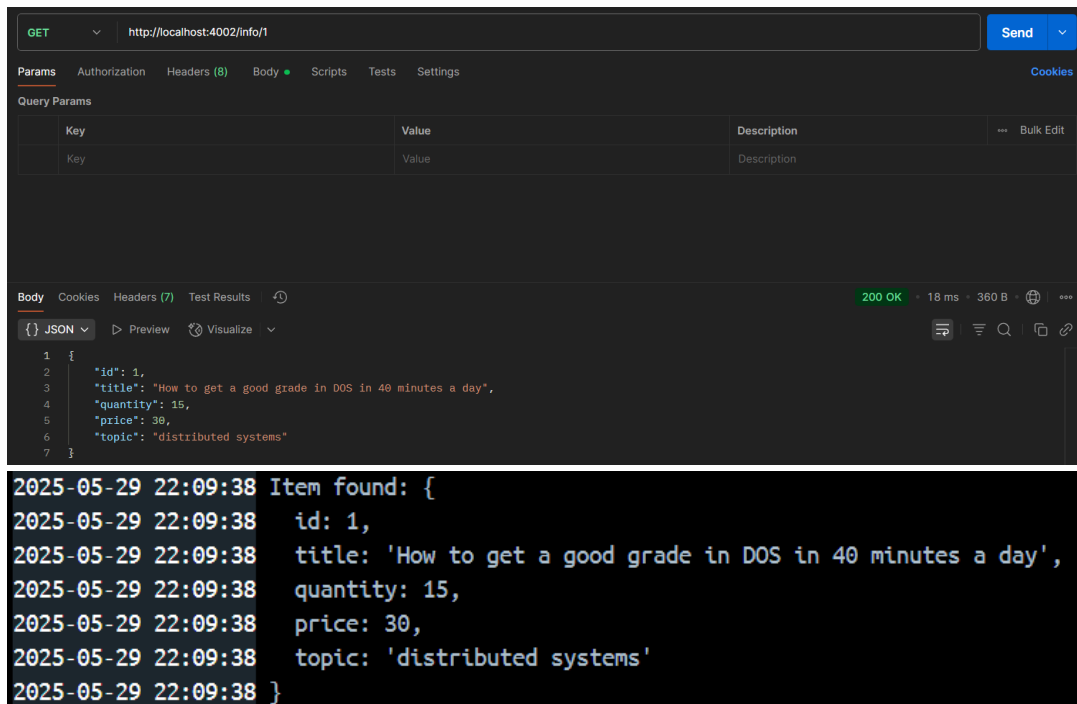
- ✓ **POST** <http://localhost:4001/update/1>
Body: {"quantity": 15, "price": 30}



```
2025-05-29 22:04:35 you are inside catalog1 other replica is http://catalog2:4000
2025-05-29 22:04:35 Cache for book 1 invalidated on frontend.
2025-05-29 22:04:35 Item updated: {
2025-05-29 22:04:35   id: 1,
2025-05-29 22:04:35   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 22:04:35   quantity: 15,
2025-05-29 22:04:35   price: 30,
2025-05-29 22:04:35   topic: 'distributed systems'
2025-05-29 22:04:35 }
```

Then check the same book in catalog2:

✓ **GET** <http://localhost:4002/info/1>



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:4002/info/1
- Status:** 200 OK
- Time:** 18 ms
- Size:** 360 B

The response body is a JSON object:

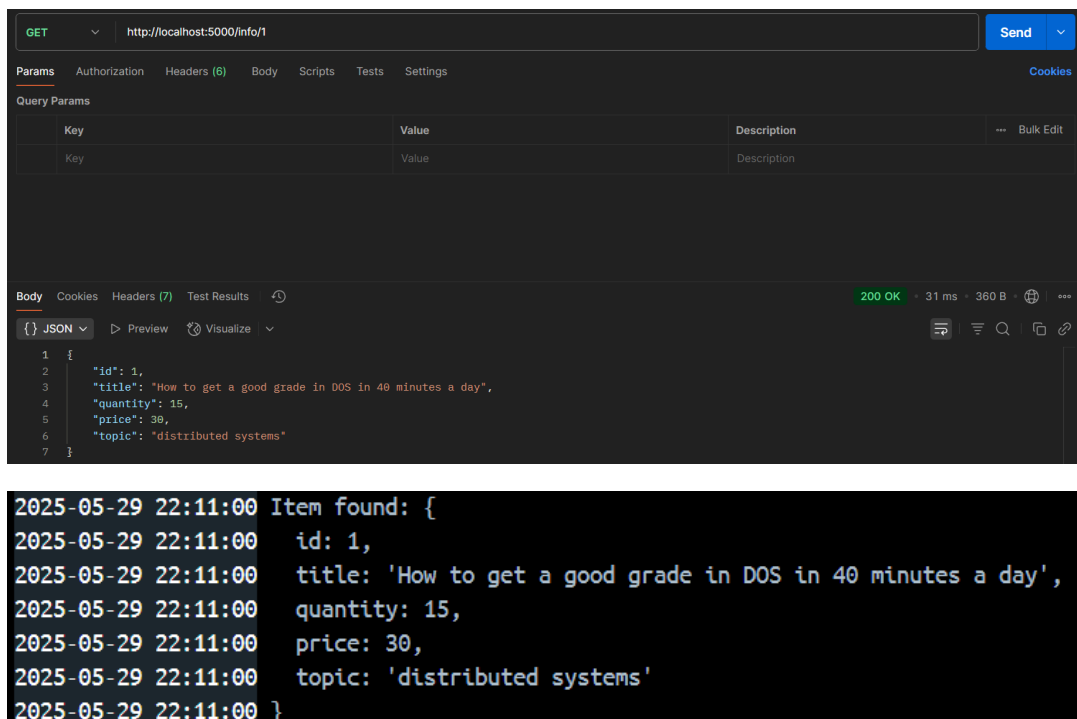
```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "quantity": 15,
  "price": 30,
  "topic": "distributed systems"
}
```

Below the screenshot, a terminal log shows the item found:

```
2025-05-29 22:09:38 Item found: {
2025-05-29 22:09:38   id: 1,
2025-05-29 22:09:38   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 22:09:38   quantity: 15,
2025-05-29 22:09:38   price: 30,
2025-05-29 22:09:38   topic: 'distributed systems'
2025-05-29 22:09:38 }
```

Here the updated quantity/price, means that the replication worked.

✓ **GET** <http://localhost:5000/info/1>



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:5000/info/1
- Status:** 200 OK
- Time:** 31 ms
- Size:** 360 B

The response body is a JSON object:

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "quantity": 15,
  "price": 30,
  "topic": "distributed systems"
}
```

Below the screenshot, a terminal log shows the item found:

```
2025-05-29 22:11:00 Item found: {
2025-05-29 22:11:00   id: 1,
2025-05-29 22:11:00   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-29 22:11:00   quantity: 15,
2025-05-29 22:11:00   price: 30,
2025-05-29 22:11:00   topic: 'distributed systems'
2025-05-29 22:11:00 }
```

Frontend server logs:

```
2025-05-29 22:11:00 [CACHE MISS] info:1
```

Test Cache Behavior

Query book information:

✓ **GET** <http://localhost:5000/info/1>

```
2025-05-28 14:49:49 Cache for book 1 invalidated on frontend.
2025-05-28 14:49:49 Item updated: {
2025-05-28 14:49:49   id: 1,
2025-05-28 14:49:49   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-28 14:49:49   quantity: 15,
2025-05-28 14:49:49   price: 30,
2025-05-28 14:49:49   topic: 'distributed systems'
2025-05-28 14:49:49 }
```

This will fetch from catalog and cache it.

Query book information again:

✓ **GET** <http://localhost:5000/info/1>

```
2025-05-28 15:10:42 Item found: {
2025-05-28 15:10:42   id: 1,
2025-05-28 15:10:42   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-28 15:10:42   quantity: 15,
2025-05-28 15:10:42   price: 30,
2025-05-28 15:10:42   topic: 'distributed systems'
2025-05-28 15:10:42 }
```

This will fetch from (cache hit).

```
[CACHE MISS] info:1
[CACHE HIT] info:1
```

Purchase the book:

✓ **POST** <http://localhost:5000/purchase/1>

```
2025-05-28 15:32:36 Cache for book 1 invalidated on frontend.
2025-05-28 15:32:36 Item updated: {
2025-05-28 15:32:36   id: 1,
2025-05-28 15:32:36   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-28 15:32:36   quantity: 14,
2025-05-28 15:32:36   price: 30,
2025-05-28 15:32:36   topic: 'distributed systems'
2025-05-28 15:32:36 }
```

invalidate the cache. (in catalog1 logs)

Query book info again:

✓ **GET** <http://localhost:5000/info/1>

```
2025-05-28 15:33:10 Item found: {
2025-05-28 15:33:10   id: 1,
2025-05-28 15:33:10   title: 'How to get a good grade in DOS in 40 minutes a day',
2025-05-28 15:33:10   quantity: 14,
2025-05-28 15:33:10   price: 30,
2025-05-28 15:33:10   topic: 'distributed systems'
2025-05-28 15:33:10 }
```

This will fetch a fresh copy from catalog, **not cached copy**. (in catalog2 logs)

Experimental Evaluation and Measurements

Measure the difference in performance when making:

- Cold requests (without cache).
- Warm requests (with cache).

We write a script that sends 100 requests of (<http://localhost:5000/info/1>), for cold tests clear the cache or restart the server before testing, and for warm tests make one request first to load the result into the cache, then test.

Request Type	Average response time	Notes
Query without cache	6.22 ms	Requests forwarded to catalog server
Query with cache	1.96 ms	Results served directly from in-memory cache

```
PS C:\Users\JC\Desktop\DOS\Bazar2\Bazar.com> node test/testPerformance.js

--- Performance Test Starting ---
Without Cache
- Average Response Time: 6.22 ms
With Cache
- Average Response Time: 1.96 ms
```

Test cache invalidation:

Step	Description	Response Time (ms)
before purchase	Cached response	37 ms
buy book	Triggers update + invalidation	47 ms
after purchase	Cache miss (pulled from catalog)	9 ms

conclusion

Our system became much faster with caching. The average response time dropped from about **6.2 ms** to just **2 ms**, which is a big improvement. This means users get results quicker, especially when they request the same book more than once.

We also tested what happens when someone buys a book. The system correctly removed the old data from the cache, so the next time the book info was requested, it pulled the fresh data from the database — just like it should.

We saw in the logs that the system sends requests to different catalog servers, which means **load balancing** is working. Overall, the system is both fast and accurate, and it handles updates correctly.